

گزارش یادگیری هفته سوم

از NoSQL تا جستجوی هوشمند و نقشه‌های تعاملی

مقدمه

در هفته سوم بوت‌کمپ، مرزهای دانش خود را از دیتابیس‌های رابطه‌ای فراتر بردیم و وارد دنیای NoSQL، موتورهای جستجوی تمام‌متن و جستجوی جغرافیایی شدیم. این هفته پلی بود میان ذخیره‌سازی داده‌های ایستا و ساخت یک سیستم واقعی، شبیه به آنچه در پلتفرم‌های آگهی (مانند دیوار) و فروشگاه‌های اینترنتی می‌بینیم. در ادامه با مفاهیم، ابزارها و معماری‌ای که فرا گرفتیم آشنا می‌شوید.

۱. MongoDB: طلوع دنیای NoSQL

تا پیش از این، تمام داده‌های خود را در قالب جدول‌هایی با ستون‌های از پیش تعیین‌شده ذخیره می‌کردیم – روشی که به آن SQL و پایگاه داده رابطه‌ای می‌گویند. اما دنیای واقعی همیشه منظم نیست. یک آگهی خودرو ویژگی‌هایی مانند مدل و کارکرد دارد، در حالی که آگهی کتاب هیچ‌کدام از این فیلدها را ندارد. MongoDB با فلسفه‌ی Schemaless (بدون طرح‌واره‌ی ثابت) این مشکل را حل می‌کند.

مفاهیم کلیدی

- Document: واحد ذخیره‌سازی در MongoDB، معادل یک رکورد در SQL، اما به شکل یک شیء JSON. هر سند می‌تواند ساختار متفاوتی داشته باشد.
- Collection: مجموعه‌ای از اسناد، شبیه به جدول در SQL، اما بدون الزام به یکسانی ستون‌ها.
- عدم نیاز به Schema: نیازی نیست از قبل مشخص کنید چه فیلدهایی وجود خواهند داشت. یک سند می‌تواند فیلد `color` داشته باشد، در حالی که سند دیگر فیلد `warranty`.

چرا MongoDB برای آگهی‌ها؟

- انعطاف‌پذیری: هر دسته‌بندی (خودرو، الکترونیک، املاک) فیلدهای خاص خود را دارد. در SQL مجبور به ایجاد ستون‌های اضافی یا جداول متعدد می‌شدیم، اما MongoDB به‌سادگی با این تنوع کنار می‌آید.
- مقیاس‌پذیری افقی: MongoDB برای توزیع داده روی چندین سرور (Sharding) طراحی شده است.
- عملکرد: عملیات CRUD روی اسناد JSON بسیار سریع و طبیعی است.

ابزارها

- MongoDB Compass: رابط کاربری گرافیکی رسمی برای کاوش داده‌ها، ساخت کوئری‌ها و دیدن ساختار اسناد.
- PyMongo: کتابخانه‌ی پایتون برای ارتباط با MongoDB، که امکان اجرای تمام عملیات را از درون کد فراهم می‌کند.

مقایسه SQL vs MongoDB

ویژگی	PostgreSQL (SQL)	MongoDB (NoSQL)
مدل داده	جداول با سطر و ستون ثابت	اسناد JSON بدون ساختار اجباری
زبان پرس‌وجو	SQL	MQL (شبیه JSON)
انعطاف‌پذیری	نیاز به تغییر طرح‌واره (ALTER TABLE)	افزودن فیلد جدید بدون تأثیر بر اسناد قبلی
روابط	JOINهای قدرتمند	ارجاع به سند دیگر (\$lookup)
موارد استفاده	داده‌های مالی، موجودی‌ها	کاتالوگ محصولات، محتوای وبلاگ، آگهی‌ها

۲. Elasticsearch: انقلابی در جستجو

جستجوی متنی با SQL (دستور `ILIKE`) برای چند هزار رکورد قابل قبول است، اما در مقیاس میلیون‌ها سند، بسیار کند و فاقد هوشمندی (مانند تصحیح غلط املایی) می‌شود. Elasticsearch یک موتور جستجوی توزیع‌شده و فوق‌العاده سریع است که بر پایه‌ی Inverted Index کار می‌کند.

Inverted Index: قلب تپنده‌ی Elasticsearch

برخلاف جستجوی خطی که تک‌تک اسناد را بررسی می‌کند، Elasticsearch پیش‌از هر جستجو، یک «نمایه‌ی معکوس» می‌سازد. فرآیند به این صورت است:

1. تجزیه (Tokenization): متن را به کلمات شکسته.
2. حذف کلمات بی‌ارزش (Stop Words): کلماتی مثل "the" و "a" حذف می‌شوند.
3. ریشه‌یابی (Stemming): کلمات به ریشه‌ی خود تبدیل می‌شوند (مثلاً "jumped" → "jump").
4. ساخت نمایه: برای هر کلمه، لیست تمام اسنادی که آن کلمه را دارند ذخیره می‌شود.

وقتی کاربر جستجویی انجام می‌دهد، به‌جای اسکن اسناد، مستقیماً به این نمایه نگاه می‌کند و در کسری از میلی‌ثانیه نتایج را باز می‌گرداند.

ویژگی‌های برتر Elasticsearch

- FullText Search: جستجوی هوشمند در میان میلیون‌ها سند در چند میلی‌ثانیه.
- Fuzzy Search: تحمل غلط‌های املایی (مثلاً جستجوی "iphone" به جای "iPhone").
- Relevance Scoring: هر نتیجه یک امتیاز (`score\_`) می‌گیرد که میزان ارتباط آن را با کوئری نشان می‌دهد.
- Aggregations: معادل `GROUP BY` در SQL – تعداد آگهی‌های هر شهر، میانگین قیمت هر دسته.
- Kibana: رابط کاربری گرافیکی برای مدیریت Elasticsearch، اجرای کوئری‌های REST و ساخت داشبورد.

Elasticsearch جایگزین دیتابیس نیست!

- یک اشتباه رایج این است که Elasticsearch را به عنوان منبع اصلی داده استفاده کنیم. Elasticsearch برای جستجو بهینه شده است، نه ذخیره سازی دائمی. معماری صحیح به این صورت است:
- PostgreSQL / MongoDB: منبع اصلی داده (ذخیره ی کامل اسناد).
- Elasticsearch: فقط فیلدهای قابل جستجو (عنوان، توضیحات، قیمت، شهر) را Index می کنیم.
- Redis: نتایج جستجوهای پرتکرار را کش می کند.

۳. جستجوی جغرافیایی (Geo Search)

قابلیت «پیدا کردن آگهی های نزدیک من» چیزی است که پلتفرم های آگهی را کاربردی می کند. Elasticsearch با نوع داده ی `geo\_point` و کوئری `geo\_distance` این امکان را فراهم می کند.

نحوه ی کار

1. ذخیره سازی مختصات: هر آگهی یک فیلد `location` از نوع `geo\_point` دریافت می کند که شامل طول (`lon`) و عرض (`lat`) جغرافیایی است.
2. کوئری فاصله: به Elasticsearch می گوییم: «تمام اسنادی که در شعاع ۱۰ کیلومتری نقطه ی (51.38, 35.74) قرار دارند را پیدا کن».
3. مرتب سازی بر اساس فاصله: نتایج به ترتیب نزدیکی (از نزدیک ترین به دورترین) بازگردانده می شوند و حتی فاصله ی دقیق هر آگهی تا نقطه ی مبدأ در پاسخ گنجانده می شود.

۴. نمایش روی نقشه با Leaflet.js

داده‌های جغرافیایی بدون نقشه بی‌روح هستند. Leaflet.js یک کتابخانه‌ی سبک و رایگان جاوااسکریپت برای نمایش نقشه‌های تعاملی است.

- با OpenStreetMap (نقشه‌ای رایگان و متن‌باز شبیه Google Maps) کار می‌کند.
- می‌توانیم Marker (نشانگر) برای هر آگهی اضافه کنیم و با کلیک روی آن، اطلاعات آگهی را در یک Popup نمایش دهیم.
- با اتصال به API خود، نقشه دیگر داده‌های تستی نشان نمی‌دهد، بلکه نتایج واقعی جستجوی جغرافیایی را به‌صورت پویا دریافت و نمایش می‌دهد.

۵. معماری یکپارچه: گردهمایی تمام فناوری‌ها

- MongoDB: ذخیره‌ی کامل آگهی‌ها با تمام جزئیات (عنوان، توضیحات، عکس‌ها، شماره تماس).
- Elasticsearch: Index کردن فیلدهای قابل جستجو (عنوان، توضیحات، شهر، دسته‌بندی، قیمت، موقعیت جغرافیایی) برای پاسخگویی سریع به جستجوها.
- Redis: کش کردن نتایج جستجوهای پرتکرار برای کاهش فشار روی Elasticsearch و MongoDB.

وقتی کاربر درخواست جستجو می‌دهد:

1. FastAPI کوئری را به Elasticsearch می‌فرستد.
2. Elasticsearch شناسه‌ی اسناد مرتبط را (به‌همراه امتیاز و فاصله) برمی‌گرداند.
3. FastAPI با استفاده از آن شناسه‌ها، داده‌های کامل را از MongoDB واکنشی می‌کند.
4. نتیجه در Redis کش می‌شود تا دفعات بعدی سریع‌تر پاسخ داده شود.

جمع بندی

در هفته سوم، ما:

- از محدودیت‌های جداول ثابت عبور کردیم و با MongoDB به استقبال داده‌های متنوع و بدون ساختار رفتیم.
- Elasticsearch را به‌عنوان یک موتور جستجوی قدرتمند در کنار دیتابیس خود نشان‌دیم و با مفاهیمی مانند Inverted Index، Fuzzy Search و Aggregation آشنا شدیم.
- قابلیت جستجوی جغرافیایی را با Elasticsearch پیاده‌سازی کردیم.
- با Leaflet.js نقشه‌ای زنده ساختیم که نتایج جستجوی مکانی را به‌صورت بصری نمایش می‌دهد.
- در نهایت، معماری ترکیبی از PostgreSQL، MongoDB، Redis و Elasticsearch را درک کردیم و دانستیم هر فناوری کجای این پازل قرار می‌گیرد..